Journal Name

Crossmark

RECEIVED
dd Month yyyy

REVISED
dd Month yyyy

PAPER

Extending the Tennessee Eastman process into a benchmark for concept drift in soft-sensor regression

Author list removed for anonymity

E-mail:

Keywords: concept drift, sensor drift, Tennessee Eastman process, benchmark data stream, soft sensor, drift detection, model adaptation

Abstract

Sample text inserted for demonstration. Replace with abstract text. Your abstract must give readers a brief summary of your article. Concisely describe the contents of your article and include key terms. It should be informative and accessible: indicate the general scope of the article and state the main results obtained and conclusions drawn. The abstract must be complete in itself: it must not contain undefined abbreviations and must not refer to any table, figure, reference or equation numbers. Normally the abstract text is not more than 300 words.

1 Introduction

Machine learning is widely applied in industrial settings, where sensor signals serve as the foundation for data-driven process modeling and optimization. In energy-intensive branches of the process industry, the operating point at which a plant is run determines how much energy is consumed per unit of product and therefore carries a direct economic weight. Optimizing this operating point requires a model that relates the variables under operational control to the resulting cost of operation. Such a model is commonly realized as a non-linear regression on recorded sensor data, for which neural networks are a frequent choice. It estimates a quantity that is not accessible to direct measurement from variables that are recorded routinely, and in this respect it belongs to the class of data-driven soft sensors that have become established in the process industry [1]. Once trained, the model is evaluated across the accessible operating range and the point of lowest cost is selected. The model therefore has to be valid across that range and not only along the trajectory the plant currently follows, which is why its training data are acquired in a designed campaign rather than taken from the historical record.

This model is based on the assumption that the relationship between the recorded variables and cost, as observed during the training phase, remains valid as long as the model is applied. However, in industrial operation, this assumption does not hold indefinitely. Plants are subject to changes due to wear and tear on mechanical components, fluctuations in the quality of raw materials, changes in catalyst activity, and changes in environmental conditions, to name a few examples. Each of these factors alters the relation the model has learned [2]. Such changes affect the process itself, so that the mapping from the recorded variables to the cost becomes time variant and the performance of a static model deteriorates over the course of its online operation. Restoring the model requires either a new experimental campaign, which occupies the plant, or a mechanism that adapts the model while production continues.

This type of deterioration is referred to as concept drift, i.e., a change over time in the joint distribution of the measured variables and the target. However, not every such change has the same effect. A shift that simply moves the observed variables to a different region of the input space leaves the mapping between them and the target variable valid, whereas a change in how the target depends on those variables invalidates this mapping. The latter case, real concept drift, is what a model cannot survive without adaptation [3, 4]. Sensor aging produces exactly this kind of shift independently of the plant, since factors such as fouling and decalibration corrupt the recorded value between limited calibration intervals, while the process itself remains unchanged.

Although a few studies have examined drift in real-world plants, among them the case of electroslag remelting [5] and a grinding plant [6], the actual onset and extent of the drift remain unknown, meaning that detection and adaptation can only be evaluated indirectly. A benchmark process avoids this limitation

since it can define the ground truth of drift already by design rather than deriving it only after the occurrence. However, data streams that offer this capability while also being publicly available and reproducible are rare [7]. Therefore, a process is required in which operating points can be specified, a drift-free reference run can be paired with the run affected by drift, and a target remains available at all times.

The Tennessee Eastman Process (TEP) is a controlled, non-linear plant simulation that meets these requirements. It defines six operating modes, provides operating cost as an explicit target, and its source code is publicly available [8–12]. Its standardized public reference data, however, is built for fault detection and decision support [13], so the one requirement it does not yet meet is the drift itself.

Based on its name, *Slow-Drift*, `IDV(13)` initially appears to be a suitable candidate. Further examination reveals, however, that it is a stationary, trendless deviation and therefore does not meet the criteria of a concept drift in the technical sense [8]. An attack point in the plant physics does not help either, since the decentralized control transfers any lasting disturbance into the manipulated variables. These are recorded alongside the measurements, and the operating cost follows deterministically from that same vector, so that the learned mapping remains valid and only the position of the data in the input space moves. The measurement chain, in contrast, lies outside this control loop, so that a drift acting on it changes what the model is given without changing the plant behind it.

This work closes that gap with the following contributions:

(i) `IDV(29)`, a multiplicative calibration drift on the stripper temperature `XMEAS(18)`, anchored in the C source of the TEP simulator;

(ii) paired nominal and drift-afflicted runs over the same sequence of operating points, together with a campaign run used as the training design;

(iii) a reference evaluation of established detectors and adaptation strategies on this stream.

The remainder of this article is structured as follows. Section 2 reviews the related work on concept drift and the TEP. Section 3 covers the necessary background. Section 4 describes how IDV(29) injects a real concept drift into the TEP. Section 5 details the generation and modeling of the resulting data streams. Section 6 presents the evaluation of established detectors and adaptation strategies on this stream, before Section 7 concludes the article.

2 Related Work

2.1 Adaptive soft sensors in the process industry

Data-driven soft sensors are a well-established tool in the process industry. They estimate a quantity that is not accessible to direct measurement from process variables that are recorded routinely, and they are derived from the historical record of the plant rather than from first principles. Kadlec et al. [1] survey the field and describe a recurring development path from the selection and cleaning of historical data through the choice of a model structure to its validation against plant data. What all of them share is the assumption that the relation learned from the historical record remains valid while the model is applied. A model used to optimize the operating point differs in this respect, since it is queried at points the plant is not currently running.

However, this assumption proves problematic in an industrial setting, since the distribution of the collected data may shift along with changing operating conditions. This phenomenon is known as concept drift, a notion established by Widmer and Kubat [14] for learning problems in which the target concept changes along with a context that is not observed. It has evolved into a distinct field of research focused on detecting changes in a data stream and adapting a model to respond to them as they occur. The process industry has encountered the same problem on the plant side, and the literature on “soft sensors” offers adaptation mechanisms as a solution to keep a deployed model aligned with the process it describes.

Kadlec et al. [2] review these mechanisms and group them into instance selection, instance weighting and ensemble methods. The recursive formulation of the partial least squares [15] is key to their calculations, and the review attributes its significance to the collinearity of the process data and the fact that a linear projection can be updated in closed-form. Kadlec and Gabrys [16] join instance weighting and ensemble methods by maintaining an ensemble of locally valid models in which the members are adapted recursively and their combination weights alongside them. Here, they report an improvement over the recursive formulation alone on data from an industrial polymerization reactor. Common to all of these mechanisms is the requirement that a reliable target value becomes available again during operation.

With regard to the measurement chain, a soft sensor should not be adapted as long as the underlying hardware sensor provides erroneous measurements [2]. A corrupted measurement is thus treated as a data quality issue rather than as a cause of drift. Nevertheless, it has been documented that the measurement hardware itself is a vulnerability. Fortuna et al. [17] report on a virtual instrument based on multilayer

perceptrons (MLP) that estimates the octane number of gasoline in a refinery. The model was deliberately designed to replace the measurement hardware during maintenance. Thus, a model whose input data deteriorates while the underlying plant remains unchanged fits between these two concepts.

2.2 Concept drift detection and adaptation in industrial applications

Gama et al. [3] provide the taxonomy and the terminology for the detection and the adaptation task, and much of the subsequent work refers to them. Outside the process industry, concept drift is studied mainly on classification tasks. Lima et al. [18] review the regression case systematically and identify 41 studies over the decade up to 2020, in which detection methods built on ensembles and neural networks prevail. Industrial soft sensing belongs to this smaller share, since the quantity a soft sensor estimates is continuous.

Concept drift has been identified in a number of industrial applications. Bayram et al. [5] investigate an electroslag remelting process at a steel manufacturer, in which a model predicts the minimum pressure of a vacuum pumping operation, while the addition of another furnace alters the conditions under which the model was trained. They address this problem using importance weighting. Zenisek et al. [19] consider the drift in the context of predictive maintenance as an indication of wear, develop their detector using synthetic data streams, and then apply it to industrial radial fans. Marijan et al. [6] model the energy consumed per ton of ground material in an industrial grinding plant with an MLP, observe the drift caused by material wear through the evolution of the model error, and compare four detection methods together with active and passive adaptation. What these real-world use cases have in common is that, while the change can be identified, it cannot be precisely determined. Moreover, there is no drift-free reference run besides the one affected, so a detected change can be classified as plausible but not as valid.

2.3 Benchmark streams with a known drift ground truth

Research on learning from data streams evaluates its methods using data streams in which the drift is defined by design. Street and Kim [20] generate data in a three-dimensional feature space, from which two features are relevant. Then, they divide the data stream into four blocks and shift the threshold separating the two classes from one block to the next. Hulten et al. [21] let the concept change continuously instead, by rotating the hyperplane that separates the classes as the stream proceeds. Both constructions state the point of change exactly, and both are classification problems whose features are drawn rather than measured. Generators of this kind are collected in the frameworks the field works with, such as MOA [22] for stream classification and clustering and river [23] for the Python ecosystem.

On the other hand, there are data streams that have been recorded on real-world systems. Souza et al. [7] collect such data streams in a public repository and investigate the associated problems, including the scarcity of publicly available non-stationary datasets. However, their collection is again tailored to classification. Kadlec et al. [2] list seven publicly available datasets of the process industry, four of which relate to batch processes or contain no target variables. Two of them describe continuous plants with a regression target, a debutanizer column and a sulfur recovery unit [24]. With approximately 10,000 measured values, the latter is the largest stream of the entire list. In none of these cases is the recorded run accompanied by a drift-free run over the same conditions, so the moment at which a change enters the data remains unknown.

However, one of the seven datasets is based on a simulator. Birol et al. [25] provide a modular simulation package for fed-batch fermentation for the production of penicillin, allowing a run to be repeated under different conditions and thus yielding a pair of runs. The process is carried out in batches, so it does not provide the stationary operating points that an optimization of the operating point refers to. This presents a dilemma. A generated data stream indicates when the concept changes, but there is no actual plant behind it, whereas a recorded data stream does involve the plant but does not specify the moment of the change. What is missing is a simulation that can be run into the same sequence of operating points twice, with the change introduced by design.

2.4 The Tennessee Eastman process as a benchmark

One simulated plant comes close to this description. Downs and Vogel [8] published the Tennessee Eastman process as a plant-wide control problem, a simulation of a chemical plant together with a catalog of disturbances that can be activated during a simulation run. Ricker [9] determined the operating points at which the plant runs at minimum cost and later proposed a decentralized control scheme that holds it at a commanded operating point [10]. Bathelt et al. [11] revised the simulation model and provided an updated implementation, and Reinartz and Enevoldsen [12] made the simulation controllable from Python through the package pyTEP, including changes to process configurations while a run is in progress. Through these contributions the process has become a standard reference plant in process monitoring research.

Russell et al. [26] detect the cataloged disturbances with canonical variate analysis and dynamic principal component analysis.Their study can thus be considered one of the works establishing this process as a reference case for statistical process monitoring. Yin et al. [27] compare the established data-driven methods against one another on the same benchmark, Seng Ng and Srinivasan [28] distribute the task over several agents that vote on a diagnosis, and Wu and Zhao [29] replace the statistical model with a deep convolutional network. The methods vary from contribution to contribution, but the research question remains the same: Which of the faults listed in the catalog is currently affecting the system? Additionally, standardized reference data for the process, designed for the same task, have been published. Reinartz et al. [13] provide a dataset for fault detection and decision support in which every run is labeled by the disturbance it carries.

A benchmark of this kind requires two properties, a sequence of operating points that can be run twice and a change introduced by design. The Tennessee Eastman process provides the first, since its operating points can be defined and commanded such that two runs differ only in what is changed intentionally. However, the alteration itself is what the process does not offer. The entries of its catalog are meant to be switched on such that a monitoring method can decide which of them is active, not to make a learned relation drift away gradually. Closing this gap is the purpose of the present work. Therefore, the list of disturbances is extended by a concept drift affecting the measurement chain, such that, across the same sequence of operating points, a drift-affected run is generated alongside a drift-free run.

3 Background

3.1 Concept drift

The deterioration of a deployed model as its underlying data changes is a well-documented phenomenon, but the terms used to describe it are not applied consistently. Occasionally the same term denotes different effects, or different terms refer to the same effect [4]. Two of them frame the present work. *Dataset shift* refers to a deviation of the data distribution between a fixed training and a test scenario [30], whereas *concept drift* refers to the evolution of that distribution over time in a continuous data stream [3]. Webb et al. [31] describe the latter as the counterpart of the former in stream learning. The type of application considered in this work falls on the seam between the two. Where the operating point of a continuously run plant is optimized, the model is identified once from a designed campaign, which is the fixed training scenario dataset shift refers to, and is then applied to the continuously recorded measurements of the running plant, which is the stream concept drift refers to. The change the model is exposed to thus occurs in a data stream, although the model itself is not learned from one, and it is this change the present work addresses. What changes in this reading is the concept, which Gama et al. [3] define as the joint distribution of the features X and the target y at a point in time t,

$$C_t = P_t(X, y). \tag{1}$$

Which part of this distribution changes, however, determines whether a model has to be adapted at all.

In its general form, a concept drift is a change of this distribution between two points in time,

$$P_t(X, y) \neq P_{t+\Delta t}(X, y). \tag{2}$$

By Bayes' theorem, $P_t(X, y) = P_t(y \mid X)\, P_t(X)$, such a change can originate in the distribution of the inputs, in the posterior that relates them to the target, or in both. A change confined to the inputs,

$$P_t(X) \neq P_{t+\Delta t}(X), \tag{3}$$

is referred to as *covariate shift* or, where the posterior is explicitly required to be unaffected, as *virtual drift*. The data then move to a different region of the input space, whereas the mapping the model was fitted to remains valid. A change of the posterior itself,

$$P_t(y \mid X) \neq P_{t+\Delta t}(y \mid X), \tag{4}$$

is referred to as *real concept drift* and occurs when the model returns an erroneous estimate for inputs it has already been trained on. According to this definition, the input distribution may shift along with the posterior or remain unchanged, $P_t(X) = P_{t+\Delta t}(X)$, the latter being referred to as *actual drift*. In either case the posterior has changed, so that the learned mapping is no longer valid and the model has to be adapted [3, 4].

A second criterion distinguishes concept drifts not by the source of the change but by the pattern in which it unfolds over time [3, 4]. A *sudden* drift replaces one concept with another at a single point in time, whereas an *incremental* drift passes through intermediate concepts, so that the transition extends over a period of time. In a *gradual* drift the old and the new concept alternate, the new one prevailing more often as the stream proceeds, and a *recurring* drift returns to a concept that has already been observed before. These patterns are to be distinguished from outliers and noise, which deviate from the current concept without replacing it and to which a detector should not respond.

3.2 Drift detection

The basic principle of a drift detector is to determine, while the stream proceeds, whether the most recent observations still originate from the same distribution as a previously defined reference. Regardless of the algorithm used, this determination amounts to a statistical hypothesis test [4]. Given a reference sample $\mathscr{D}_t$ and a sample $\mathscr{D}_{t+w}$ collected over a window of w observations, the null hypothesis states that both originate from the same distribution,

$$H_0 : P(\mathscr{D}_t) = P(\mathscr{D}_{t+w}) \quad \text{against} \quad H_1 : P(\mathscr{D}_t) \neq P(\mathscr{D}_{t+w}). \tag{5}$$

This is assessed by a test statistic $S(\mathscr{D}_t, \mathscr{D}_{t+w})$ that quantifies the deviation between the two samples and is compared against a threshold λ. As long as $S \leq \lambda$ holds, the null hypothesis is retained and the stream is treated as stationary. Once the threshold is exceeded, it is rejected and a drift is declared. The threshold thus determines the trade-off that every detector faces, given that lowering the threshold shortens the delay until a change is detected but, at the same time, increases the number of false alarms.

The detectors differ in terms of the sample on which the test is performed, which is how they are typically categorized [4]. *Data distribution-based* detectors apply a distance measure directly to the feature distributions of two windows of the stream [32]. They require no labels, but they respond to any change of the input distribution, whether or not the predictive performance of the model is affected by it [3]. *Performance-based* detectors apply the test to the error of the model instead, so that a drift is declared only where the deterioration it causes becomes measurable. *Multiple hypothesis-based* detectors combine several such tests, either in parallel through a decision rule or hierarchically through a warning level that a second test confirms [33], and *context-based* detectors extend them by information about the system or the model.

This paper focuses on the performance-based family. It requires the target value to remain available during operation, so that the model's error can be evaluated at every step. The choice is determined by the task itself, because a model used to select an operating point should be adjusted wherever its estimate deviates from the true value. Since a detector compares only specific windows of the data stream rather than its entirety, a deliberate change to the operating point constitutes a change in the observed input distribution, even if the system and the learned mapping remain unchanged. However, a deterioration in the measurement chain shifts the measured variables as well. Monitoring the input distribution thus responds to both, while only the latter invalidates the learned mapping. Referring the test to the model error distinguishes the two.

Within the performance-based family, the detectors are distinguished by the strategy with which they track the error [4]. In its simplest form, a performance indicator is compared against a fixed threshold over a moving window, without a statistical test and without a warning level [3]. Methods of *statistical process control* monitor the error rate instead and signal a drift once it departs by a statistically defined margin from the best value observed so far, whereas *window-based* methods maintain a reference and a detection window over the error stream and compare the two by a statistical test. *Ensemble-based* methods combine several learners and derive the indication of a change from the development of their individual errors. The evaluation presented here comprises a threshold on the root mean squared error as a reference, DDM and EDDM as representatives of statistical process control, and ADWIN and KSWIN as representatives of the window-based methods.

Both methods of statistical process control considered here monitor a binary error signal, so that a regression error has to be converted by a threshold beforehand. DDM tracks the error rate p_i and its standard deviation σ_i under a normal approximation of the underlying binomial distribution. It retains the smallest sum $p_{\min} + \sigma_{\min}$ observed so far as its reference, issues a warning once $p_i + \sigma_i$ exceeds this reference by twice $\sigma_{\min}$ and declares a drift at three times $\sigma_{\min}$ [34]. EDDM replaces the error rate with the distance between two consecutive errors, which shortens as a drift develops gradually. The current mean distance is related to the largest one observed so far, and a warning and a drift are declared once this ratio falls below two respective thresholds [35].

The two window-based methods considered differ in how the two windows are formed. ADWIN keeps a single window of the error stream and enlarges it as long as the stream appears stationary. The window is repeatedly split into two consecutive subwindows, whose means are compared against a bound derived from Hoeffding's inequality. Once the difference exceeds that bound, a drift is declared and the older part of the window is discarded, so that the window length itself reflects the extent of the current concept [36]. KSWIN, in contrast, keeps a window of fixed size, from which it forms a detection window of the most recent observations and a reference window drawn uniformly at random from the older part. Both are compared by the Kolmogorov-Smirnov test, which makes no assumption about the shape of the underlying distribution [37].

3.3 Drift adaptation

A detector establishes that the concept has changed, but does not modify the model. Restoring the predictive performance requires the model to be adapted, for which the survey of Gama et al. [3] remains the reference. Every adaptation strategy is characterized by two decisions: one regarding the data and one regarding the model. The data on which the adaptation is performed are stored in a short-term memory. This short-term memory typically consists of a window covering the most recent observations, along with a mechanism that governs how older observations are discarded. Forgetting quickly increases the responsiveness to a change, but it also increases the susceptibility to noise. The model, on the other hand, serves as the long-term memory in which the relation observed in the data is generalized.

Regarding the long-term memory, there are two approaches to adapting the model. Full retraining involves discarding the current model and training a new one based on the data stored in the short-term memory. This serves as a robust reference in research but requires these data to be sufficient on their own, a demanding requirement, particularly where they have to be gathered from a design of experiments. Incremental learning instead updates the existing model with the observations as they arrive, so that no new model is trained. Every update is subject to the stability-plasticity dilemma, the trade-off between incorporating new knowledge and retaining what has already been learned [3]. Too much plasticity leads to catastrophic forgetting, whereas too much stability renders the adaptation ineffective.

The second characteristic of an adaptation strategy is its trigger. A passive, or blind, strategy updates the model continuously, irrespective of whether a change has occurred. It requires no detector, but it spends effort on updates that may not be needed and keeps the model in motion even if the concept is stable. In contrast, an active, or informed, strategy performs an update only once a detector has signaled a drift. The adaptation is thereby targeted, but it becomes dependent on the quality of the detection, since a change that goes unnoticed triggers nothing. Combining both is intended to resolve this conflict, by retaining the periodic update as a fallback for changes the detector misses [3].

3.4 The Tennessee Eastman process

The TEP is a non-linear simulation of an industrial chemical plant, in which two liquid products are formed from four gaseous reactants in four irreversible and exothermic reactions, accompanied by an inert component and a by-product [8]. It comprises five main units connected in sequence, as shown in figure 1. The reactants are fed into the reactor, where they react over a non-volatile catalyst held in the liquid phase, and the reaction heat is removed by an internal cooling bundle. The products leave the reactor as vapor, are cooled in the condenser and are separated from the remaining gases in the vapor-liquid separator, which divides the process into a forward product path and a recycle loop. In the stripper, the liquid is freed from residual reactants by the gas fed in below, whereas the uncondensed vapors are returned to the reactor by the compressor, with a purge preventing the inert component and the by-product from accumulating.

The process involves 41 measured variables and 12 manipulated variables, most of which operate valves [8]. In its original form, the system is unstable in an open-loop control configuration and triggers a safety shutdown shortly after startup. Downs and Vogel deliberately leave the control strategy undefined and present its design as a task for the benchmark. They do define six operating modes based on the mass ratio of the two products and the production rate. A mode, however, represents a requirement rather than an operating point, and the process can meet it in many different steady states. Ricker determines, for each mode, the steady state that meets this requirement at the lowest operating costs [9], and derives a decentralized control scheme that maintains the system at a specified operating point and rejects disturbances [10]. In this scheme, the manipulated variables are specified by the controllers and merely recorded, while the plant is operated using setpoints whose combination defines the operating point, shown in gray in figure 1 and listed in table 2. The operating costs are calculated by the simulation itself and are thus available at every step of a simulation run.

Alongside the plant, Downs and Vogel provide a catalog of 20 process disturbances that can be activated during a run, expanded to 28 in the revision by Bathelt et al. [8, 11]. Its entries represent typical fault cases, among them step changes in the feed composition, random variations of a feed temperature and a sticking valve. Each of them is switched on at a point in time and is present from then on, so that a monitoring method can decide which one is active. A slow and continuous change of the kind considered for concept drift is not part of the catalog. The simulation is available as source code and can be driven from Python through pyTEP [11, 12].

4 Injecting real concept drift into the Tennessee Eastman process

Among the disturbances of the catalog, `IDV(13)` appears to come closest to the purpose of this work, since it is labeled a *slow drift* of the reaction kinetics [8]. Once it is activated, the simulation multiplies the rates of the two primary reactions, which form the liquid products, by the time-varying factors `r1f` and `r2f`. At random intervals, each factor draws a new target value uniformly from a bounded interval of $[0.75, 1.25]$

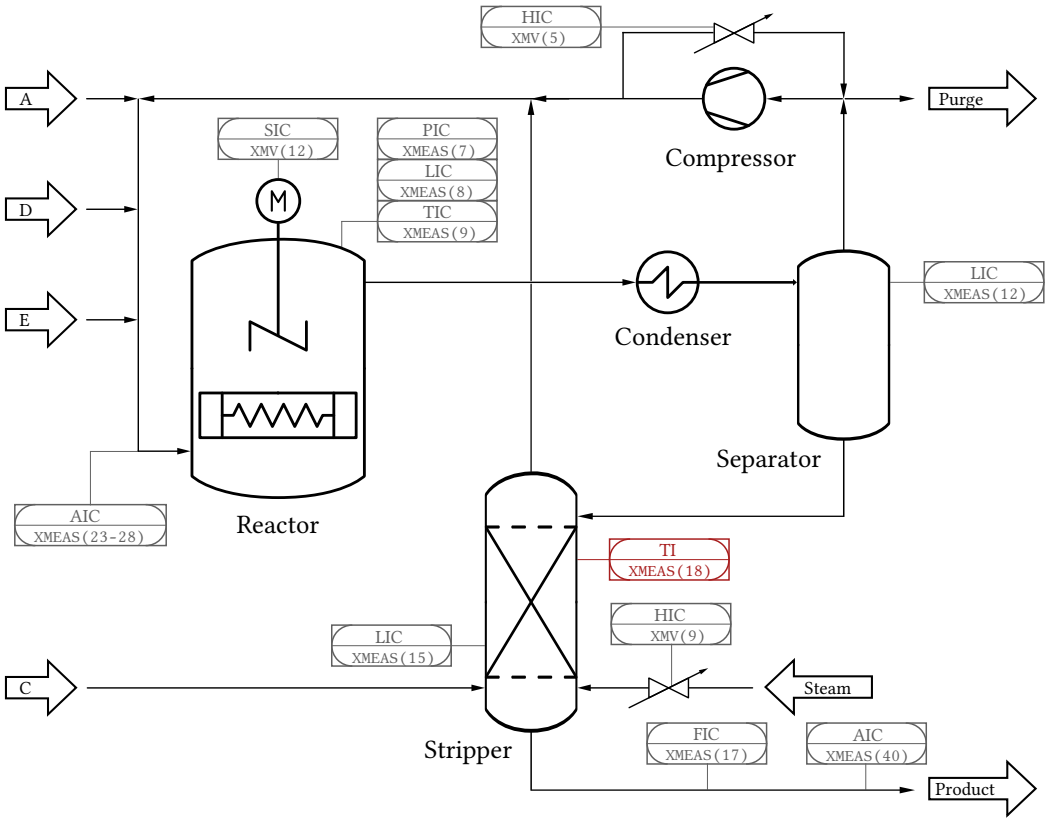


Figure 1. Flowsheet of the TEP with its five main units, after [8].

around its nominal value of 1 and approaches it by cubic interpolation, so that its course is smooth but strictly bounded. The memory of this course therefore extends no further than the next target value, which lies only a few hours ahead. The resulting fluctuation causes a one-time expansion of the distribution of the affected process variables once the disturbance is activated. However, it does not shift the distribution any further, and the process remains stationary and without a trend. Since a concept drift according to (2) requires the joint distribution to change over time, `IDV(13)` does not constitute a concept drift despite its label, but a disturbance of stationary character.

The simulation is therefore extended by a further disturbance, `IDV(29)`, which is activated and scaled through the same interface as its other entries. Regarding the place where the drift may occur, an impact on the physics of the system—such as the reaction kinetics discussed in `IDV(13)`—seems plausible, since it represents an actual change in the process behavior. The TEP, however, is fully controlled, and its control scheme converts any persistent disturbance into a lasting change of the manipulated variables, which are recorded along with the measured ones. The operating costs, in turn, are computed by the simulator from this very measurement vector. A change in the system's physics thus remains visible in the observed variables, and every quantity the simulator derives therefrom is calculated based on the changed values themselves. Thus, its relationship to these variables is preserved. What shifts is the position of the data within the domain of the observed variables, which, according to (3), constitutes a covariate shift.

However, the sensors that provide the measured values are located outside this chain of effects. In industrial practice, sensors are subject to fouling and aging, while maintenance and calibration intervals are limited. It is therefore assumed that a sensor provides increasingly distorted values over a period of weeks, even though the process itself remains unchanged. `IDV(29)` reproduces this kind of sensor deterioration on the stripper temperature `XMEAS(18)`, highlighted in red in figure 1, which is chosen for three properties. It is recorded by the simulation, it is not part of any control loop, and it does not enter the calculation of the operating costs. The drift therefore distorts only the value that is output, while the process value, and thus the behavior of the plant, remains unaffected.

`IDV(29)` realizes this deterioration as a multiplicative calibration error and scales the output value of `XMEAS(18)` by the factor $1 + \kappa_m\, c(t)$, in which κ_m denotes the drift coefficient and $c(t)$ the drift signal. Since the factor acts multiplicatively, the absolute error of the measurement depends on the operating point at which the plant is run. The operating costs, taken as the target y, continue to follow the unchanged process, whereas the recorded temperature departs increasingly from it. Over the course of the drift, the same observed value thus corresponds to different operating costs, so that the posterior $P(y \mid X)$ shifts and

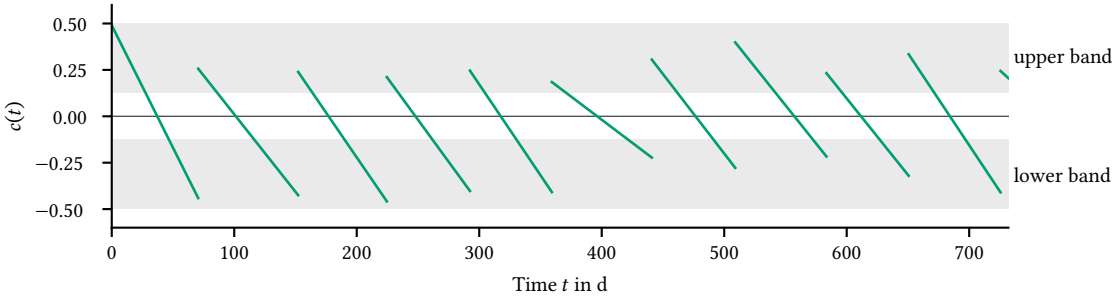


Figure 2. Drift signal $c(t)$ of `IDV(29)`.

Table 1. Parameters of `IDV(29)`.

Symbol	Description	Value
κ_m	Drift coefficient	0.3
T_{seg}	Nominal segment duration	73 d
γ_τ	Relative shift of the segment boundaries	0.2
c_i^+	Start level, drawn uniformly from	$[0.125, 0.5]$
c_i^-	Target level, drawn uniformly from	$[-0.5, -0.125]$

the drift can be classified according to (4) as a real concept drift. However, as the distorted measurement itself is one of the recorded variables, the input distribution shifts as well, so that the drift does not constitute an actual drift.

The drift signal $c(t)$ determines how this error evolves over time and is modeled according to a sensor found in real-world applications. Between maintenance intervals, fouling and aging cause the error to change continuously, while recalibration resets it immediately. The simulated period is therefore divided into segments whose nominal duration T_{seg} represents the calibration interval. Since a maintenance date varies in practice, the inner segment boundaries $\tau_1 < \tau_2 < \cdots$ are displaced randomly from this grid by a fraction γ_τ of the segment duration. Within the i-th segment, the signal falls linearly from a start level c_i^+, drawn from an upper band, to a target level c_i^-, drawn from a lower band,

$$c(t) = c_i^+ + \frac{c_i^- - c_i^+}{\tau_i - \tau_{i-1}}\,(t - \tau_{i-1}), \qquad t \in \left[\tau_{i-1},\, \tau_i\right), \tag{6}$$

and at each boundary it jumps to the start level of the following segment. Figure 2 shows the resulting course of $c(t)$ over the simulated period. Each segment appears as a separate line that falls from the upper to the lower shaded band, and the gaps between the lines mark the sudden drifts at the segment boundaries. This sawtooth combines most of the patterns introduced in section 3.1. Within a segment, the concept passes through intermediate states, which constitutes an incremental drift, whereas at a boundary it is replaced at once, which constitutes a sudden drift. As the signal crosses zero in every segment, where the measurement is undistorted, the original concept is revisited repeatedly, so that the drift is recurring as well. A gradual drift, in which two concepts alternate, does not occur.

Table 1 lists the parameters of `IDV(29)`. The segment duration of 365 d/5 represents five maintenance events per year. This results in a relatively large number of operating points per calibration interval, so that the calibration error changes only slightly when an operating point is reached. Consequently, each operating point reflects a single drift value. The shift fraction keeps adjacent segment boundaries apart while still breaking the periodicity of the grid. The drift coefficient is bounded from above by the requirement that the peak of the calibration error remains below the spread of the stripper temperature caused by the process itself. With the chosen value, the recorded temperature deviates from the process value by at most 14.7 %, or about 9.2 K.

Since the Simulink model integrates the plant with the solver `ode45`, whose step size varies, the model function is called at points in time that do not lie on a fixed grid. The drift signal is therefore formulated as a closed-form expression in the simulation time t and evaluated anew at every call. The displacement of each boundary and the two levels of each segment are derived deterministically from a seed and the segment index, so that the signal carries no state from one call to the next. The signal is thus independent of the step size the solver chooses, and repeated runs with the same seed carry an identical drift. Only the number of segments follows from the simulated duration.

The intervention in the source code of the simulator is confined to a few lines of `temexd_mod.c`,

Listing 1. Application of the sensor drift in `temexd_mod.c`, abridged and simplified.

```
xmeas[17] = tcc;                        /* process value of XMEAS(18) */
if (idv[29] > 0.) {
    if (drift_tonset < 0.) drift_tonset = time;
    driftf = 1. + idv[29] * drift_signal(time - drift_tonset,
                              DRIFT_PERIOD, DRIFT_GAMMA, DRIFT_SEED);
} else {
    drift_tonset = -1.;
    driftf = 1.;
}
xmeas[DRIFT_SENSOR_INDEX] *= driftf;
```

shown in listing 1. The drift signal itself is implemented in the separate file `drift_function.c`, which the S-function includes as source and which can also be compiled into a shared library, so that the identical signal can be evaluated from Python. In the listing, `tcc` denotes the stripper temperature computed by the process model, which is first assigned to the zero-indexed measurement vector `xmeas` at the position of `XMEAS(18)`. The entry `idv[29]` holds the value the disturbance is set to through the interface, which is the drift coefficient κ_m. On the first call after activation, the simulation time `time` is stored in `drift_tonset`, which is reset to a negative value once the disturbance is switched off, so that a renewed activation restarts the pattern. The function `drift_signal` returns $c(t)$ for the time elapsed since activation and is parameterized by the constants `DRIFT_PERIOD`, `DRIFT_GAMMA` and `DRIFT_SEED`, which correspond to T_{seg}, γ_τ and the seed of the random levels. The resulting factor `driftf` scales the entry at `DRIFT_SENSOR_INDEX`, which refers to `XMEAS(18)`, only after the process value has been assigned. It therefore acts solely on the recorded measurement, while the simulated process behaves exactly as in the undisturbed case. Both files, together with the adjustments to pyTEP required to run the extended simulation, are publicly available in a fork of the package [38].

5 Data stream generation and modeling

The data stream is generated with pyTEP, which runs the Simulink model and controls it from Python [12]. Between resets of the simulator, setpoints are ramped, disturbances are activated and the simulation is advanced interval by interval, after which all recorded variables and the operating costs are returned as `pandas` DataFrames and appended to a continuous record. As part of the compiled model, `IDV(29)` is activated like any other disturbance, so that a drift-affected run differs from a nominal one only in this setting. The pyTEP package itself requires only minor extensions, which are included in the fork.

The operating points of the stream are organized around the six operating modes of the TEP. Their cost-optimal steady states are listed in Table 2, where Reinartz et al. [13] retain only those variables from the full set of variables specified by Ricker [9] that are controlled in the decentralized control scheme [10]. The levels, the reactor pressure, the steam valve and the agitator speed are fixed across all modes and therefore shown in gray. The modes thus differ in production rate, product composition, reactor temperature and feed composition, the latter given by the molar fraction y_{AC} of A and C in the reactor feed and the share y_{A} of A therein. Two of them additionally differ in the position of the recycle valve. Six isolated operating points neither cover the range a model for operating point selection should remain valid for, nor do they vary the setpoints independently, so that their individual effects on operating costs cannot be separated. Each mode is therefore extended to a series of adjacent points using a design of experiments (DOE).

The design builds on the dataset of Reinartz et al. [13], in which individual setpoints are varied within ±15 % of their nominal value in steps of 5 %. This window is applied, but the setpoints are varied jointly rather than individually, and the grid is replaced by a sample from a continuous range of values. The bounds are relative to the nominal value of the respective mode, so that the deflection remains comparable across setpoints of different magnitudes. Only the agitator speed, which is already at the upper limit of its range, is not raised any further. Beyond that, the window is deliberately kept narrow, since a combination far from the optimum can trip one of the safety shutdowns of the TEP. Within the window, each mode is sampled by a Latin hypercube, which covers every setpoint axis evenly regardless of the number of points. Using a maximin criterion, the design is selected from a larger number of independent draws as the one whose smallest distance between two points is largest, which avoids clusters that would leave part of the operating region unoccupied. Each mode receives its own design, complemented by the unchanged nominal setpoints, which yields 50 points per mode and 300 candidates in total.

A candidate from the design is not necessarily feasible, so each one is first run in isolation, starting from the steady state of mode 1 in which the simulator is initialized. All twelve setpoints are ramped to their target values rather than stepped, since a step drives the controllers to their limits and can trigger a

Table 2. Setpoints of the six operating modes after [13].

Setpoint	Unit	Mode 1	Mode 2	Mode 3	Mode 4	Mode 5	Mode 6
Production rate	$m^3\,h^{-1}$	22.89	22.73	18.04	36.04	23.55	20.20
Stripper level	%	50	50	50	50	50	50
Separator level	%	50	50	50	50	50	50
Reactor level	%	65	65	65	65	65	65
Reactor pressure	kPa	2800	2800	2800	2800	2800	2800
G in product	mol-%	53.80	11.66	90.09	53.35	11.65	90.07
y_A	%	63.137	64.196	62.110	61.940	64.030	61.468
y_{AC}	mol-%	51.00	54.24	47.43	58.76	54.32	48.79
Reactor temperature	°C	122.9	124.2	121.9	128.2	124.6	123.0
Recycle valve	%	1	1	77.62	1	1	71.166
Steam valve	%	1	1	1	1	1	1
Agitator speed	%	100	100	100	100	100	100

safety shutdown. The point is then held until the transients have decayed and the process has settled in a quasi-stationary state. These isolated runs are recorded and later form the pool from which the training data are drawn.

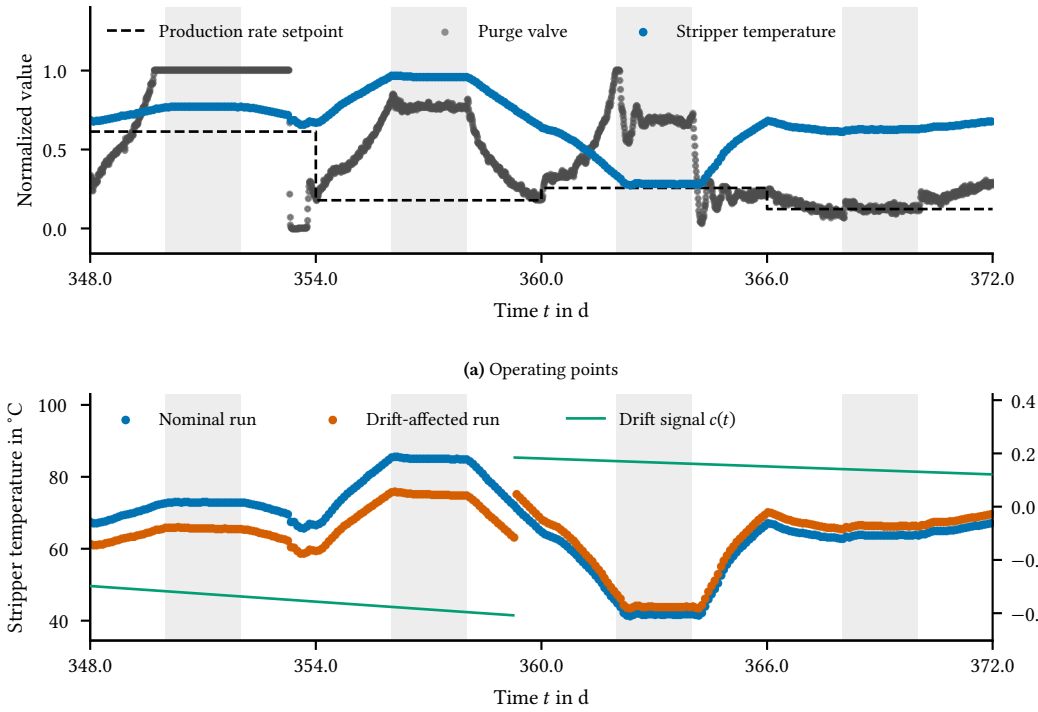
The feasible points are subsequently strung together into a continuous stream. Between two of them, the setpoints are ramped back to the steady state of mode 1 over another ramp of the same duration, so that every point is approached from the same conditions. The order of the points within the stream is drawn at random. In the continuous run, a few of them still shut the simulation down, so these points are removed and the run is repeated, which leaves an uninterrupted stream of 122 operating points covering about 2 years of plant operation. The fault run repeats exactly this sequence with `IDV(29)` active and starts from a freshly initialized simulator, so that both runs begin in the same state. Since the calibration drift does not touch the process, the two are identical on the process side and differ only in the recorded stripper temperature. Three datasets thus result: the isolated points as the pool for training, the nominal run as a drift-free reference, and the fault run as the drift-affected stream.

Figure 3 shows an excerpt of the stream over four operating points. Panel (a) relates the setpoint of the production rate to two recorded variables, the stripper temperature and the purge valve. It shows the ramp into the point, the shaded holding window with its quasi-stationary course, and the return to the initial state. Panel (b) shows the stripper temperature for both runs along with the drift signal, demonstrating how the drift affects the recorded signal. Both runs follow the same pattern imposed by the operating points, while the drift-affected one departs from the nominal by an offset that follows the drift signal, including its jump at a segment boundary.

The data stream is modeled as a regression of a few recorded variables on the operating costs. The model is trained on the campaign and then applied to the nominal and the fault run, which differ only in the recorded stripper temperature. Any deviation between their error courses is therefore attributable to the drift, and comparing the two reveals the deterioration in the model. The model label is the operating costs, which the simulator computes deterministically from the measurement vector according to the calculation given by Downs and Vogel [8]. The features are derived from that same calculation, which is dominated by the losses through the purge stream. Selected are the variables that govern this stream, namely the purge valve as its rate, the feed of component E to the reactor as its source and the separator temperature as the split between purge and condensate, complemented by the controlled mass ratio of the two products, which separates the six operating modes. Variables entering the cost calculation directly are excluded, so that the model learns the relation within the process rather than reproducing the arithmetic of the simulator.

Another feature added is the stripper temperature. Its impact on costs is based on a forward selection process, considering variables successively according to their contribution to improving the model's quality. At the same time, it carries the calibration error into the model, while process and costs remain untouched, so that the difference between the two error courses is attributable to the model alone. Among the features, it is moreover the one that responds most sensitively to a distortion of its own measurement.

The DOE spans the operating region, whereas operation is not restricted to the points it contains. Therefore, training uses the quasi-stationary holding intervals of a subset of the feasible operating points, referred to as the campaign. It contains the nominal setpoints of the six modes, while its remaining points are selected by the same maximin criterion as the design. The feasible points outside the campaign carry the assessment on operating points the model has not seen. For data preprocessing, the signals are scaled by a min-max normalization and subsequently smoothed by a moving average. The normalization is



(a) Operating points

(b) Drift influence

Figure 3. Excerpt of the generated data stream over four operating points.

Table 3. Parameters of the data stream and of the model.

Data stream		Model	
Points per mode	50	Campaign points	60
Candidates in total	300	Smoothing window in samples	10
Ramp duration	2 d	Hidden layers	2
Holding duration	2 d	Neurons per layer	12
Block duration	6 d	Maximum epochs	200
		Batch size	200

determined on all isolated operating points and applied unchanged to the nominal and the fault run. However, no outlier filter is applied, since the spread across the operating points carries the signal and the extreme points mark the border of the covered region.

The model is a multilayer perceptron with tanh activation in its hidden layers and a linear output neuron, trained with the Adam optimizer on the mean squared error. Despite the size of the process, this level of complexity meets the requirements, since it involves the regression of several features on a scalar target value, which is evaluated exclusively in steady-state conditions. The dynamics of the process are not taken into account. Since the number of operating points is small, the ramp-up sections of the campaign points are used during validation, so that all quasi-steady-state areas can be used for training. An early stopping on the validation error terminates the training and falls back on the weights of the best epoch. Table 3 summarizes the model parameters along with those of the stream.

6 Evaluation and results

6.1 Drift-induced model degradation

The model is evaluated based on its simulation error, i.e., the deviation between the operating costs calculated by the simulator and the model's estimate. The evaluation is performed over the entire course of both the nominal run and the fault run, including the ramps between operating points that are not part of the training data. The drift signal $c(t)$ serves as a reference. Since the calibration drift does not affect the process, the ratio of the recorded stripper temperatures from both runs corresponds to the calibration factor.

Figure 4 shows the normalized simulation error for both runs over the first six months of the stream, along with the drift signal $c(t)$. During nominal operation, it fluctuates around zero, while its variance

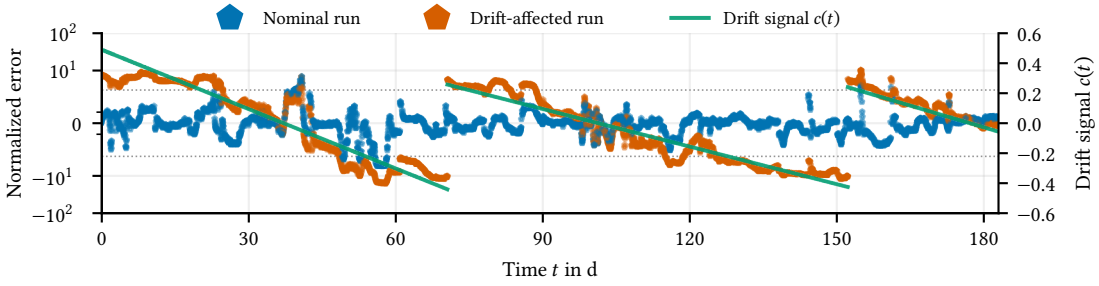


Figure 4. Normalized simulation error of the nominal and the fault run.

Table 4. Simulation error of the nominal and the fault run.

	RMSE		Error
Operating points	nominal	fault	ratio
Campaign	0.010	0.060	6.22
Remaining	0.015	0.054	3.56
All	0.013	0.057	4.34

changes from one operating point to the next, thereby replicating the block structure of the stream. In the fault run, the error deviates from the nominal error as the drift signal progresses. Within each segment, it initially lies above the nominal error, decreases along with $c(t)$, and falls below it as soon as the signal changes sign, until the jump at the segment boundary restores the initial offset. The drift thus remains visible in the error curve despite the variation between operating points.

Table 4 lists the RMSE of both runs for the operating points of the campaign, for the remaining points of the stream and for all points together. The RMSE is given as a fraction of the cost range of the isolated operating points, on which the min-max normalization is determined. The error ratio relates the RMSE of the fault run to that of the nominal run and thus quantifies the degradation caused by the drift. Across all operating points, the RMSE increases markedly from the nominal to the fault run. Across all operating points, the RMSE increases markedly from the nominal to the fault run. As can be seen in figure 4, the offset changes its sign within each segment, so that the drift raises the error in proportion to the magnitude of $c(t)$ rather than shifting it as a whole.

6.2 Drift detection

The detectors introduced in section 3.2 monitor the simulation error of the model on the fault run. It is normalized with the mean and the standard deviation of the error on the nominal run, so that the nominal run sets the scale against which a change has to become visible. KSWIN, ADWIN and the threshold detector process its absolute value as a continuous signal. DDM and EDDM, in contrast, require the binary signal described in section 3.2, for which an error is counted as a misprediction once its absolute value exceeds the threshold λ_{bin}. Due to normalization, this threshold corresponds to a multiple of the standard deviation of the nominal error and is adjusted together with the other parameters of the two detectors.

7 Conclusion

Acknowledgments
Removed for anonymity

Funding
Removed for anonymity

Author contributions
Removed for anonymity

Data availability
Removed for anonymity

Supplementary data
Sample text inserted for demonstration.

References

[1] Petr Kadlec, Bogdan Gabrys, and Sibylle Strandt. "Data-driven soft sensors in the process industry." In: *Computers & Chemical Engineering* 33.4 (2009), pp. 795–814. DOI: 10.1016/j.compchemeng.2008.12.012.

[2] Petr Kadlec, Ratko Grbić, and Bogdan Gabrys. "Review of adaptation mechanisms for data-driven soft sensors." In: *Computers & Chemical Engineering* 35.1 (2011), pp. 1–24. DOI: 10.1016/j.compchemeng.2010.07.034.

[3] João Gama et al. "A survey on concept drift adaptation." In: *ACM Computing Surveys* 46.4 (2014), pp. 1–37. DOI: 10.1145/2523813.

[4] Firas Bayram, Bestoun S. Ahmed, and Andreas Kassler. "From concept drift to model degradation: An overview on performance-aware drift detectors." In: *Knowledge-Based Systems* 245 (2022), p. 108632. DOI: 10.1016/j.knosys.2022.108632.

[5] Firas Bayram et al. "A drift handling approach for self-adaptive ml software in scalable industrial processes." In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 2022, pp. 1–5.

[6] Pascal Marijan et al. "Concept Drift in Industrial Material Processing." In: *2025 11th International Conference on Control, Decision and Information Technologies (CoDIT)*. Vol. 1. IEEE. 2025, pp. 647–652. DOI: 10.1109/CoDIT66093.2025.11321558.

[7] Vinicius M. A. Souza et al. "Challenges in benchmarking stream learning algorithms with real-world data." In: *Data Mining and Knowledge Discovery* 34.6 (2020), pp. 1805–1858. DOI: 10.1007/s10618-020-00698-5.

[8] J. J. Downs and E. F. Vogel. "A plant-wide industrial process control problem." In: *Computers & Chemical Engineering* 17.3 (1993), pp. 245–255. DOI: 10.1016/0098-1354(93)80018-I.

[9] N. L. Ricker. "Optimal steady-state operation of the Tennessee Eastman challenge process." In: *Computers & Chemical Engineering* 19.9 (1995), pp. 949–959. DOI: 10.1016/0098-1354(94)00043-N.

[10] N. Lawrence Ricker. "Decentralized control of the Tennessee Eastman challenge process." In: *Journal of Process Control* 6.4 (1996), pp. 205–221. DOI: 10.1016/0959-1524(96)00031-5.

[11] Andreas Bathelt, N. Lawrence Ricker, and Mohieddine Jelali. "Revision of the Tennessee Eastman process model." In: *IFAC-PapersOnLine* 48.8 (2015), pp. 309–314. DOI: 10.1016/j.ifacol.2015.08.199.

[12] Christopher Reinartz and Thomas T. Enevoldsen. "pyTEP: A Python package for interactive simulations of the Tennessee Eastman process." In: *SoftwareX* 18 (2022), p. 101053. DOI: 10.1016/j.softx.2022.101053.

[13] Christopher Reinartz, Murat Kulahci, and Ole Ravn. "An extended Tennessee Eastman simulation dataset for fault-detection and decision support systems." In: *Computers & Chemical Engineering* 149 (2021), p. 107281. DOI: 10.1016/j.compchemeng.2021.107281.

[14] Gerhard Widmer and Miroslav Kubat. "Learning in the presence of concept drift and hidden contexts." In: *Machine Learning* 23.1 (1996), pp. 69–101. DOI: 10.1023/A:1018046501280.

[15] S. Joe Qin. "Recursive PLS algorithms for adaptive data modeling." In: *Computers & Chemical Engineering* 22.4–5 (1998), pp. 503–514. DOI: 10.1016/S0098-1354(97)00262-7.

[16] Petr Kadlec and Bogdan Gabrys. "Local learning-based adaptive soft sensor for catalyst activation prediction." In: *AIChE Journal* 57.5 (2011), pp. 1288–1301. DOI: 10.1002/aic.12346.

[17] Luigi Fortuna et al. "Virtual instruments based on stacked neural networks to improve product quality monitoring in a refinery." In: *IEEE Transactions on Instrumentation and Measurement* 56.1 (2007), pp. 95–101. DOI: 10.1109/TIM.2006.887331.

[18] Marilia Lima et al. "Learning under concept drift for regression—a systematic literature review." In: *IEEE Access* 10 (2022), pp. 45410–45429. DOI: 10.1109/ACCESS.2022.3169785.

[19] Jan Zenisek, Florian Holzinger, and Michael Affenzeller. "Machine learning based concept drift detection for predictive maintenance." In: *Computers & Industrial Engineering* 137 (2019), p. 106031. DOI: 10.1016/j.cie.2019.106031.

[20] W. Nick Street and YongSeog Kim. "A streaming ensemble algorithm (SEA) for large-scale classification." In: *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2001, pp. 377–382. DOI: 10.1145/502512.502568.

[21] Geoff Hulten, Laurie Spencer, and Pedro Domingos. "Mining time-changing data streams." In: *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2001, pp. 97–106. DOI: 10.1145/502512.502529.

[22] Albert Bifet et al. "MOA: Massive online analysis, a framework for stream classification and clustering." In: *Proceedings of the First Workshop on Applications of Pattern Analysis*. PMLR. 2010, pp. 44–50.

[23] Jacob Montiel et al. "River: machine learning for streaming data in Python." In: *Journal of Machine Learning Research* 22.110 (2021), pp. 1–8.

[24] Luigi Fortuna et al. *Soft Sensors for Monitoring and Control of Industrial Processes*. Advances in Industrial Control. London: Springer, 2007. DOI: 10.1007/978-1-84628-480-9.

[25] Gülnur Birol, Cenk Ündey, and Ali Çinar. "A modular simulation package for fed-batch fermentation: penicillin production." In: *Computers & Chemical Engineering* 26.11 (2002), pp. 1553–1565. DOI: 10.1016/S0098-1354(02)00127-8.

[26] Evan L. Russell, Leo H. Chiang, and Richard D. Braatz. "Fault detection in industrial processes using canonical variate analysis and dynamic principal component analysis." In: *Chemometrics and Intelligent Laboratory Systems* 51.1 (2000), pp. 81–93. DOI: 10.1016/S0169-7439(00)00058-7.

[27] Shen Yin et al. "A comparison study of basic data-driven fault diagnosis and process monitoring methods on the benchmark Tennessee Eastman process." In: *Journal of Process Control* 22.9 (2012), pp. 1567–1581. DOI: 10.1016/j.jprocont.2012.06.009.

[28] Yew Seng Ng and Rajagopalan Srinivasan. "Multi-agent based collaborative fault detection and identification in chemical processes." In: *Engineering Applications of Artificial Intelligence* 23.6 (2010), pp. 934–949. DOI: 10.1016/j.engappai.2010.01.026.

[29] Hao Wu and Jinsong Zhao. "Deep convolutional neural network model based chemical process fault diagnosis." In: *Computers & Chemical Engineering* 115 (2018), pp. 185–197. DOI: 10.1016/j.compchemeng.2018.04.009.

[30] Joaquin Quiñonero-Candela et al. *Dataset Shift in Machine Learning*. Cambridge, MA: MIT Press, 2009. DOI: 10.7551/mitpress/9780262170055.001.0001.

[31] Geoffrey I. Webb et al. "Characterizing concept drift." In: *Data Mining and Knowledge Discovery* 30.4 (2016), pp. 964–994. DOI: 10.1007/s10618-015-0448-4.

[32] Daniel Kifer, Shai Ben-David, and Johannes Gehrke. "Detecting change in data streams." In: *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*. 2004, pp. 180–191.

[33] Jie Lu et al. "Learning under concept drift: A review." In: *IEEE Transactions on Knowledge and Data Engineering* 31.12 (2019), pp. 2346–2363. DOI: 10.1109/TKDE.2018.2876857.

[34] João Gama et al. "Learning with drift detection." In: *Advances in Artificial Intelligence – SBIA 2004*. Vol. 3171. Lecture Notes in Computer Science. 2004, pp. 286–295. DOI: 10.1007/978-3-540-28645-5_29.

[35] Manuel Baena-García et al. "Early drift detection method." In: *Fourth International Workshop on Knowledge Discovery from Data Streams*. 2006, pp. 77–86.

[36] Albert Bifet and Ricard Gavaldà. "Learning from time-changing data with adaptive windowing." In: *Proceedings of the 2007 SIAM International Conference on Data Mining*. 2007, pp. 443–448. DOI: 10.1137/1.9781611972771.42.

[37] Christoph Raab, Moritz Heusinger, and Frank-Michael Schleif. "Reactive soft prototype computing for concept drift streams." In: *Neurocomputing* 416 (2020), pp. 340–351. DOI: 10.1016/j.neucom.2019.11.111.

[38] Anonymous. *pyTEP fork with the concept drift disturbance IDV(29)*. Version 1.0.0-paper. TODO: after acceptance replace author, url and add doi (Zenodo). 2026. URL: https://anonymous.4open.science/r/TODO (visited on 09/16/2026).